



**VNiVERSiDAD
D SALAMANCA**

Facultad de Ciencias

Grado en Ingeniería Informática

TRABAJO FIN DE GRADO

Desarrollo de una IA para jugar Riichi Mahjong

Development of an AI to Play Riichi Mahjong

Predicción y evaluación estratégica de descartes de Riichi Mahjong mediante Transformers

Prediction and Strategic Evaluation of Riichi Mahjong Discards Using Transformers

ANEXO III

Especificación de Diseño

Autor

Julia Ruiperez de Brito

Tutor

Prof. Vidal Moreno Rodilla

Departamento

Informática y Automática

Salamanca

Junio 2026

Índice general

1	Introducción	2
2	Arquitectura General del Sistema	2
2.1	Flujo General de Datos	2
2.2	Organización Modular	2
3	Diseño y Representación de los Datos	3
3.1	Decodificación y Normalización	3
3.2	Representación Común	3
3.3	Tokenización	4
3.4	Embeddings	4
4	Diseño del Modelo	5
4.1	Arquitectura Transformer	5
4.2	Entrada y Salida del Modelo	5
5	Diseño del Entrenamiento y Evaluación	6
5.1	Pipeline de Entrenamiento	6
5.2	Evaluación Tradicional	6
5.3	Evaluación Estratégica	7
6	Trazabilidad de Requisitos	8

1 Introducción

Este documento recoge la especificación de diseño del sistema desarrollado durante este Trabajo Fin de Grado. En él se describe la arquitectura general del sistema, la representación de los datos, el diseño del modelo y los procesos utilizados para su entrenamiento y evaluación.

El objetivo de este anexo es describir cómo se han diseñado los diferentes componentes del sistema y cómo se relacionan entre ellos para cumplir con los requisitos definidos en el Anexo II. Para ello, se presenta el flujo de los datos desde su obtención a partir del dataset de Tenhou hasta la generación y evaluación de las predicciones realizadas por el modelo.

2 Arquitectura General del Sistema

2.1 Flujo General de Datos

La arquitectura del sistema se organiza como un conjunto de componentes encargados de transformar progresivamente los datos originales hasta obtener una predicción del modelo y su posterior evaluación.

Primeramente, los datos son obtenidos del dataset de Tenhou y decodificados mediante el parser. La instantánea resultante se normaliza en una representación común para todos los ejemplos del dataset. A partir de esta representación, el tokenizer genera los tokens correspondientes a las diferentes categorías de información del estado.

Los tokens son transformados posteriormente en embeddings vectoriales que sirven como entrada al Transformer Encoder. El modelo procesa esta información y genera una predicción sobre el descarte a realizar. Finalmente, las predicciones pueden ser procesadas por el sistema de evaluación para obtener las diferentes métricas utilizadas durante el proyecto.

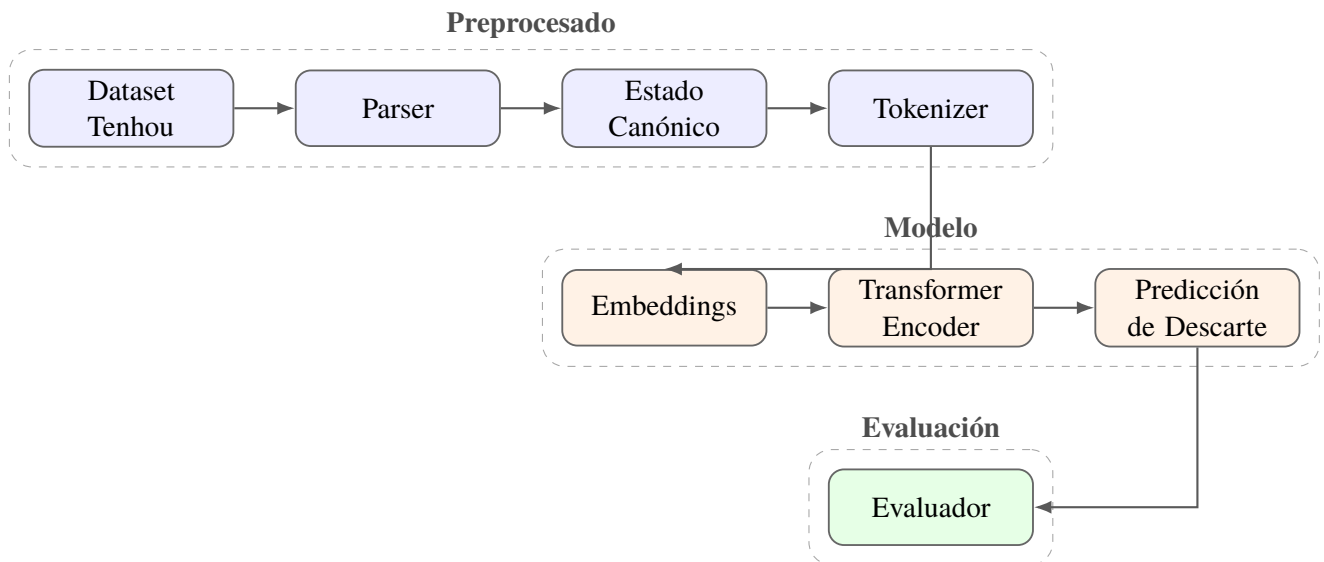


Figura 1: Arquitectura general del sistema y flujo de datos entre sus principales componentes.

2.2 Organización Modular

El sistema se diseñó de forma modular, separando las diferentes etapas del procesamiento de los datos, el modelo y su evaluación. De esta forma, cada componente tiene una

responsabilidad concreta y puede ser modificado sin necesidad de realizar grandes cambios en el resto del sistema.

Primeramente, el parser se encarga de decodificar la información original del dataset y generar una representación normalizada de la instantánea. A continuación, el tokenizer transforma esta información en los tokens utilizados por el modelo. La representación vectorial de estos tokens se realiza mediante los embeddings antes de ser procesados por el Transformer Encoder.

Por otro lado, el entrenamiento y la evaluación se mantienen separados del procesamiento de los datos y de la definición del modelo. Esta separación permite utilizar un mismo modelo en diferentes experimentos y modificar las métricas de evaluación sin afectar al proceso de entrenamiento.

Esta organización también facilita la incorporación de nuevas representaciones, modelos o métodos de evaluación durante las diferentes iteraciones del proyecto.

3 Diseño y Representación de los Datos

3.1 Decodificación y Normalización

Los datos utilizados para el entrenamiento se obtienen a partir del dataset derivado de registros de Tenhou. Cada muestra almacena una instantánea de la información disponible en el momento de una decisión.

Para utilizar estos datos durante el entrenamiento, se diseñó un parser encargado de descomprimir, decodificar y normalizar la información almacenada. El resultado contiene, entre otros campos, la mano del jugador, los descartes, las combinaciones abiertas, los indicadores de Dora, la puntuación de los jugadores y las acciones válidas. No se reconstruye el historial completo de la partida ni información oculta.

A partir de esta decodificación se genera una representación común de la instantánea, utilizada posteriormente por el resto de componentes del sistema. De esta forma, el procesamiento posterior no depende directamente del formato original utilizado por el dataset.

3.2 Representación Común

Una vez decodificada la instantánea original, esta se transforma a una representación común. El objetivo es disponer de una estructura organizada para todos los ejemplos utilizados durante el entrenamiento y la evaluación.

El estado canónico agrupa la información general de la partida, la información propia del jugador, el estado de los cuatro jugadores y las acciones disponibles. Esta representación sirve como punto intermedio entre los datos originales de Tenhou y el sistema de tokenización.

De forma simplificada, la estructura utilizada es la siguiente:

```
{  
  "round_wind": ...,  
  "num_honba": ...,  
  "num_riichi": ...,  
  "player_wind": ...,  
  "position": ...,  
  "remain_tiles": ...,
```

```

    "dora_indicators": [...],
    "hand_tiles": [...],

    "players": {
        "0": {...},
        "1": {...},
        "2": {...},
        "3": {...}
    },

    "valid_actions": [...],
    "action_idx": ...
}

```

A partir de esta estructura, el tokenizer puede acceder de forma directa a cada categoría de información y generar los tokens correspondientes sin depender del formato original del dataset.

3.3 Tokenización

A partir del estado canónico, se realiza un proceso de tokenización para transformar la información del estado de juego en una secuencia que pueda ser procesada por el modelo Transformer.

Cada token representa una categoría concreta de información del estado. De esta forma, se mantienen separados elementos como las piezas de la mano, los indicadores de Dora, los vientos, la posición del jugador o la información correspondiente a cada uno de los jugadores.

La tokenización permite representar un estado complejo de Riichi Mahjong mediante una secuencia estructurada de elementos. Además, facilita la incorporación o modificación de categorías de información sin necesidad de cambiar la representación original de los datos.

Una vez generada la secuencia de tokens, cada uno de ellos se transforma posteriormente en una representación vectorial mediante el sistema de embeddings.

3.4 Embeddings

Una vez generados los tokens, estos deben ser transformados a una representación numérica que pueda ser procesada por el Transformer. Para ello, el sistema utiliza embeddings que representan cada token mediante un vector de valores continuos.

Los embeddings permiten que las diferentes categorías de información del estado compartan un mismo espacio de representación antes de ser procesadas por el modelo. Estos vectores son aprendidos durante el entrenamiento, de forma que el modelo puede aprender representaciones útiles para realizar la predicción del descarte.

La representación final de cada token se obtiene mediante la suma de seis embeddings correspondientes al tipo de token, la pieza, su copia, el jugador, el tipo de acción y su posición absoluta dentro de la secuencia.

Aunque los objetos intermedios conservan otros atributos, la tensorización utilizada en los experimentos no los transmite al modelo. Por tanto, campos como puntuaciones, estados de Riichi, Honba, número de palos de Riichi, fichas restantes, marcas de *tsumogiri* o detalles

completos de las combinaciones no cuentan con embeddings propios en la implementación evaluada.

Por lo tanto, los embeddings funcionan como la conexión entre la representación semántica generada por el tokenizer y la entrada numérica utilizada por el Transformer Encoder.

4 Diseño del Modelo

4.1 Arquitectura Transformer

El modelo desarrollado utiliza una arquitectura basada en un Transformer Encoder. Esta arquitectura fue escogida debido a que el objetivo del modelo no es generar una secuencia de elementos, sino procesar la información completa de un estado de juego y realizar una predicción de clasificación sobre el descarte a realizar.

La entrada del modelo está formada por los embeddings generados a partir de los tokens del estado. Estos son procesados por las diferentes capas del Transformer Encoder, permitiendo al modelo aprender las relaciones existentes entre las diferentes partes del estado de juego.

La representación obtenida por el Transformer pasa posteriormente por una capa de clasificación. Para ello, se seleccionan las representaciones correspondientes a los tokens de las acciones candidatas y se genera una puntuación o *logit* para cada una de ellas. Finalmente, la acción con mayor puntuación es seleccionada como la predicción realizada por el modelo.

Se escogió una arquitectura relativamente sencilla con el objetivo de disponer de un modelo base sobre el que realizar las diferentes iteraciones y experimentos del proyecto. De esta forma, el trabajo se centra principalmente en estudiar la capacidad de un Transformer para aprender las decisiones realizadas por jugadores reales a partir de la representación diseñada.

4.2 Entrada y Salida del Modelo

La entrada del modelo corresponde a la secuencia de tokens generada a partir del estado de juego. Cada uno de estos tokens es transformado en un embedding antes de ser procesado por el Transformer Encoder.

A partir de esta entrada, el modelo genera una serie de logits que representan la puntuación asignada a cada acción candidata presente en el estado. Estos valores no representan directamente probabilidades, sino la preferencia aprendida por el modelo para cada una de las acciones disponibles.

Durante la predicción, se selecciona como acción aquella que presenta el logit más alto. Durante el entrenamiento, estos logits se comparan con la acción realizada por el jugador real mediante una función de pérdida de entropía cruzada.

De esta forma, el modelo aprende a partir de los ejemplos del dataset a asignar una mayor puntuación a las decisiones observadas durante las partidas reales.

5 Diseño del Entrenamiento y Evaluación

5.1 Pipeline de Entrenamiento

El entrenamiento del modelo se realiza mediante aprendizaje por imitación utilizando los estados y descartes obtenidos del dataset de Tenhou. Cada ejemplo de entrenamiento está formado por un estado de juego y la acción realizada por el jugador real en dicho estado.

Primeramente, los estados son procesados mediante el parser y el sistema de tokenización. A continuación, los datos son agrupados en batches y enviados al modelo para realizar la predicción. Los logits generados se comparan con las acciones reales mediante la función de pérdida de entropía cruzada.

A partir de esta pérdida se realiza la actualización de los parámetros del modelo mediante backpropagation. Este proceso se repite durante las diferentes épocas de entrenamiento.

La infraestructura permite evaluar un conjunto de validación y guardar checkpoints durante el entrenamiento. Sin embargo, estas posibilidades no se utilizaron de la misma forma en todas las ejecuciones: el experimento de 500.000 estados se entrenó sobre el bloque inicial disponible y los dos checkpoints finales se compararon después sobre un conjunto independiente de 52.563 estados.

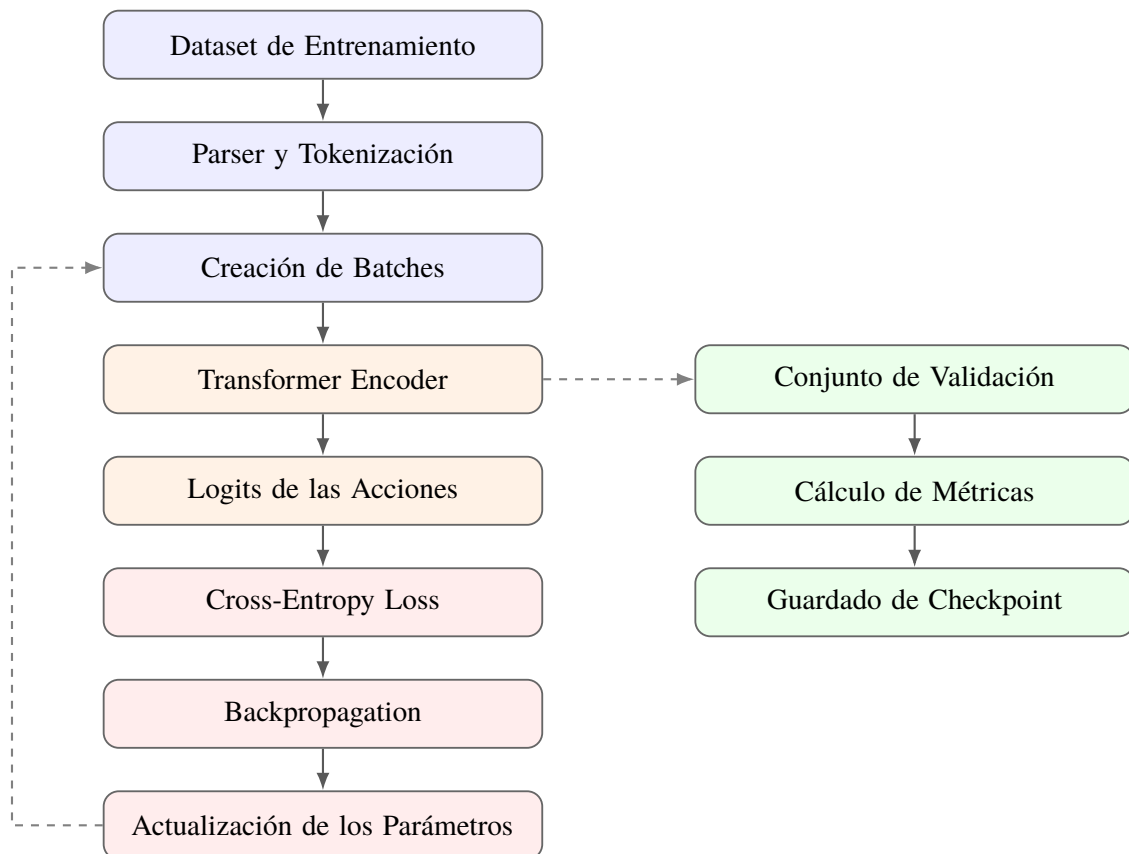


Figura 2: Pipeline de entrenamiento, validación y almacenamiento de checkpoints del modelo.

5.2 Evaluación Tradicional

Inicialmente, la evaluación del modelo se diseñó utilizando métricas tradicionales de clasificación. La métrica principal utilizada fue la exactitud (*accuracy*), que permite medir

el porcentaje de estados en los que el descarte predicho por el modelo coincide exactamente con el descarte realizado por el jugador real.

Además, se utilizaron métricas de *Top-k Accuracy*. Estas permiten comprobar si el descarte realizado por el jugador se encuentra entre las acciones con mayor puntuación asignadas por el modelo, aunque no corresponda exactamente con su primera predicción.

Estas métricas permitieron analizar la evolución del modelo durante el entrenamiento y comparar diferentes iteraciones. Sin embargo, una predicción diferente a la realizada por el jugador no implica necesariamente una decisión estratégicamente incorrecta. A partir de esta limitación surgió la necesidad de desarrollar un método de evaluación adicional basado en conceptos propios del Riichi Mahjong.

5.3 Evaluación Estratégica

A partir de las limitaciones encontradas en las métricas tradicionales, se diseñó un sistema de evaluación estratégica. El objetivo de este sistema es analizar las predicciones incorrectas del modelo y determinar si realmente representan una peor decisión desde el punto de vista del juego.

Primeramente, se compara el descarte predicho por el modelo con el descarte realizado por el jugador. Si ambos coinciden, la decisión se considera una coincidencia exacta. En caso contrario, se calcula el Shanten resultante de ambos descartes y se compara su progreso hacia una mano completa.

Cuando ambos descartes mantienen el mismo Shanten, se utiliza el Ukeire para comparar la cantidad de piezas que permiten mejorar la mano después de cada decisión. A partir de estas comparaciones, las predicciones son clasificadas en diferentes categorías estratégicas.

Las principales categorías utilizadas son:

- `exact_match`: el modelo realiza el mismo descarte que el jugador.
- `equivalent`: ambos descartes producen una situación estratégicamente equivalente.
- `near_equivalent`: existe una pequeña diferencia entre ambos descartes, pero se consideran decisiones cercanas.
- `better_ukeire`: la predicción del modelo mantiene el mismo Shanten, pero obtiene un mejor Ukeire.
- `better_shanten`: la predicción del modelo obtiene un mejor Shanten.
- `ukeire_loss`: la predicción mantiene el mismo Shanten, pero produce una pérdida significativa de Ukeire.
- `shanten_loss`: la predicción del modelo produce un peor Shanten.

A partir de estas categorías se obtiene una medida de precisión estratégica, considerando no solamente las coincidencias exactas, sino también aquellas decisiones que pueden considerarse válidas desde el punto de vista estratégico.

Este sistema no pretende determinar de forma absoluta si una decisión es correcta o incorrecta. Su objetivo es proporcionar más información sobre el comportamiento del modelo y permitir analizar qué conceptos estratégicos ha conseguido aprender.

6 Trazabilidad de Requisitos

Finalmente, se establece una relación entre los requisitos funcionales definidos en el Anexo II y los diferentes componentes diseñados para cumplirlos. Esta trazabilidad permite comprobar que cada una de las funcionalidades requeridas se encuentra contemplada dentro del diseño del sistema.

Cuadro 1: Trazabilidad entre requisitos funcionales y componentes del diseño

ID	Requisito	Componente de diseño
RF-01	Reconstrucción de estados de juego	Parser y normalización de la instantánea
RF-02	Generación de una representación canónica	Estado canónico
RF-03	Transformación del estado en tokens semánticos	Tokenizer
RF-04	Generación de embeddings	Capa de embeddings
RF-05	Entrenamiento mediante aprendizaje por imitación	Pipeline de entrenamiento y modelo Transformer
RF-06	Predicción de descartes	Transformer Encoder y capa de clasificación
RF-07	Evaluación mediante métricas tradicionales	Evaluator tradicional
RF-08	Evaluación mediante Shanten y Ukeire	Evaluator estratégico
RF-09	Comparación entre diferentes modelos	Sistema de evaluación y análisis de resultados

A partir de esta relación, podemos comprobar que los requisitos funcionales definidos para el sistema están representados por uno o varios componentes dentro de su diseño.